Technical Report

Hospital Appointment and Patient Management System

Salih Çelik 21058011, Muhammet Al 22058607, Yunus Özgül 21058062, Ahmet Tuğrul Aki 22058011, Ömer Özyurt 20058060

GitHub Repository Link: https://github.com/salihcozer/Hospital-Management-System.git

1. Introduction and Problem Formulation

Contemporary healthcare administration systems frequently encounter substantial operational bottlenecks due to highly fragmented data architectures. The fundamental issue lies in the structural disconnect between scheduling infrastructure and dynamic clinical records. Furthermore, patients possess highly variable and unstructured medical attributes—such as differing arrays of allergies, blood types, or genetic markers. Attempting to model these dynamic attributes within rigid, traditional relational columns often results in severe data sparsity (excessive NULL values) and inflexible schemas. Consequently, this fragmentation induces scheduling anomalies, impedes rapid access to critical medical history during emergencies, and obstructs the systematic analysis of departmental workloads.

2. Project Rationale and Strategic Objectives

The primary objective of this project is to architect a highly cohesive, normalized relational database that mitigates data redundancy while ensuring strict referential integrity. This initiative is critically important as it directly correlates with both institutional efficiency and patient safety. By amalgamating traditional relational structural constraints with modern, schema-less JSON storage capabilities, the proposed architecture offers a hybrid solution. It maintains the rigid transactional reliability (ACID properties) required for appointment scheduling while introducing the flexibility necessary to accommodate complex, nested clinical data without necessitating continuous schema migrations.

3. Architectural Schema Design To ensure comprehensive data coverage while maintaining strict adherence to normalization principles, the core database schema has been designed with seven highly interrelated entities:

1. **departments:** Functions as a standardized lookup table for clinical departments.
2. **doctors:** Associates medical professionals with their respective departments via foreign key constraints.
3. **patients:** Houses primary demographic data alongside a designated JSON column allocated for variable clinical attributes.
4. **appointments:** Operates as the central junction entity mapping patients to doctors, enforced by rigorous temporal (DATETIME) and state-based (CHECK) constraints.
5. **medical_records:** Archives diagnostic outcomes exclusively linked to finalized appointments.
6. **medications:** A normalized repository of pharmaceutical assets and dosages.

7. **prescriptions:** A many-to-many resolution entity correlating specific medical records to prescribed medications.

4. Core Database Functionalities & Analytical Prototyping

The system architecture is designed to implement and demonstrate key relational paradigms: **Domain Constraints (CHECK), Performance Optimization (B-Tree Indexing), Data Abstraction (Views), Semi-structured Data Integration (JSON), and Recursive/Analytical Processing (CTEs).**

Prototype 1: Data Abstraction for Daily Provider Schedules (Views)

SQL

```
CREATE VIEW v_todays_appointments AS
SELECT
    d.last_name AS Doctor,
    p.full_name AS Patient,
    a.appointment_datetime AS Time,
    a.status AS Status
FROM appointments a
JOIN doctors d ON a.doctor_id = d.doctor_id
JOIN patients p ON a.patient_id = p.patient_id
WHERE DATE(a.appointment_datetime) = DATE('now');
```

Prototype 2: Document-Style Querying for Emergency Diagnostics (JSON Extraction)

SQL

```
SELECT full_name, json_extract(medical_attributes, '$.blood_type') as Blood_Type
FROM patients
WHERE json_extract(medical_attributes, '$.allergies') LIKE '%Penisilin%';
```

Prototype 3: Hierarchical Aggregation for Workload Analytics (CTEs)

SQL

```
WITH DepartmentWorkload AS (
    SELECT d.name, COUNT(a.appointment_id) as total_appointments
    FROM appointments a
    JOIN doctors doc ON a.doctor_id = doc.doctor_id
    JOIN departments d ON doc.department_id = d.department_id
    GROUP BY d.name
```

```
)
SELECT * FROM DepartmentWorkload
ORDER BY total_appointments DESC LIMIT 3;
```

6. Methodological Approach to Data Simulation To conduct rigorous empirical testing of the schema, query execution plans, and indexing strategies, a procedural data generation script was engineered utilizing the Python Faker library. This methodological approach circumvents the inherent statistical biases and scale limitations of manual data entry, successfully provisioning a high-cardinality, relational synthetic dataset (encompassing thousands of logically consistent records) to simulate a high-traffic production environment.